

ラボ 2.4: フィールドバス プロトコルの探索

1. はじめに

1.1. 必要なリソース

- ControlThings プラットフォーム VM
- Velocio Ace11 PLC

1.2. まとめ

このラボでは、Velocio PLCをControlThings VMIに接続し、いくつかの簡単な手法を使用して、そこに送信されるModbus RTUシリアルトラフィックを監視します

1.3. 目標

- フィールドバス通信に接続されたシステム上のシリアルトラフィックをキャプチャする方法を学びます
- Modbus RTU シリアルトラフィックを手動で分析する方法を学習します。

2. PLCを接続する

このラボを実施するには、PLCでDesktop\Velocio PLC\ChemicalMixerSFC-Ace11-Completed\ChemicalMixerSFC.vioロジックを実行している必要があります。前回の1.3「HMIプログラミング」ラボを既に完了しており、それ以降PLCロジックを変更していない場合は、PLCで正しいロジックが既に実行されているはずです。そうでない場合は、[「HMIプログラミング」ラボの再プログラミングセクション](#)に進み、PLCをプログラミングしてください。

ControlThings Platform VMを起動する

☰ PLCをControlThingsプラットフォームに接続する



- 入力スイッチボード上のすべてのスイッチをオフにします（数字から離れた場所）。
- 入力スイッチボードを PLC の左側に接続します。
- USB ケーブルを使用して PLC をコンピューターに接続します。
- PLCをControlThingsプラットフォームに仮想的に接続するようにVMwareを構成する

🔥 この4つのステップで迷っていませんか?

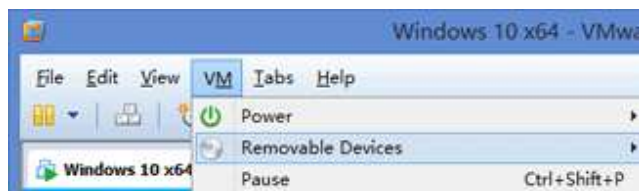
☰ PLCの左側に入力スイッチボードを接続します



USBケーブルを使用してPLCをコンピューターに接続します

PLCをVMに仮想的に接続

これを行うには、VMwareのメインメニューを開き、「**VM**」の下にある「**Removable Devices**」オプションを選択します。ここに、VMに仮想的に接続できるUSBデバイスのリストが表示されます。Velocio ACE 11 PLCは、VMwareでは**Luminary Micro**デバイスとして表示されるはずです。



VMware Fusion では、[仮想マシン]の[USB]の下に同様のオプションがあります。

× デバイスのリストに PLC が表示されませんか?

まず、ホストシステムが PLC を認識できるかどうかを確認しましょう。

- **Windowsユーザー：** PLCを接続せずに**デバイスマネージャー**を開き、表示されるデバイスをメモ（またはスクリーンショット）してください。その後、PLCを接続して新しいデバイスが追加されていることを確認してください。おそらく、汎用デバイスまたは認識されないデバイスとして表示される可能性があります、これは問題なく、想定内の動作です。
- **Mac ユーザー:** PLC を接続し、**システム情報ユーティリティ**を開いて、USB セクションで**製造元が VelocioNetworks**であるデバイスを探します。

ホストコンピュータがPLCを認識できない場合は、別のコンピュータで同じテストをお試しください。どちらのコンピュータでもPLCを認識できない場合は、TA、SME、または講師にご連絡ください。

ホストがPLCを認識している限り、問題はVMwareまたはVirtualBoxのインストールに関係している可能性があります。以下の手順を段階的に試し、いずれかの方法が機能するまでお試しください。

1. PLCを別のUSBポートに接続してみてください
2. ノートパソコン全体を再起動してみてください。VMware サービスの再起動が必要な場合があります。
3. VMのUSBセクションで、USB 3.0サポートを有効にするか無効にするなど、さまざまな構成を試してください。
4. VMwareの最新バージョンを使用していることを確認してください。最新バージョンでない場合は、最新バージョンへのアップグレードをお試しください。ハードウェアやOSのアップデートを常に最新の状態にしておくと、VMwareがお使いのデバイスをサポートする適切なUSBホストブリッジドライバを提供していない場合があります。

Velocio PLC が ControlThings プラットフォームに接続されていることを確認するには、`lsusb` ターミナルからのコマンドを使用できます。

2.1. USB デバイスの一覧表示

PLCが接続されていることを確認する

ターミナルを開く

入力

```
lsusb
```

✓ 成功

```
Bus 001 Device 002: ID 0e0f:000b VMware, Inc.  
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub  
Bus 002 Device 005: ID 1cbe:0002 Luminary Micro Inc.  
Bus 002 Device 004: ID 0e0f:0008 VMware, Inc.  
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub  
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse  
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
```

Luminary Micro Inc. のエントリが表示されていれば、接続されています。バス、デバイス、IDの値は、上記の私のものとは異なる場合があります

このラボの後半では、PLCがどのUSBバスに接続されているかを知る必要があります。これは `lsusb` コマンド出力で確認できます。上記の例では、PLCはバス002のデバイス005であることがわかります。

PLCが接続されているバス番号を書き留めます

2.2. PLCの動作確認

PLC が適切にプログラムされ、プロセスが期待どおりに実行されていることを確認しましょう。

PLCスイッチ1を ON 位置

PLCの6つの出力LEDが異なるパターンで点滅し始めるはずですが、表示されるパターンは以下のとおりです。

状態	バッチタイマー	出力LEDオン
1	0～2秒	●○○○○○
2	2～4秒	○●●●○○
3	4～6秒	○○●●○○
4	6～10秒	○○○●○○
5	10～15秒	○○○○●○

3. USBトラフィックをキャプチャする

シリアル通信やフィールドバス通信をキャプチャする最も簡単な方法は、通信に参加しているコンピュータでWiresharkなどのツールを使用することです。Wiresharkなどのツールは、通信カードのドライバに入出力されるデータをキャプチャすることでこれを実現します。シリアル通信やフィールドバス通信の場合、ドライバはほとんどの場合、単純なシリアルデバイスとして表現されます。WindowsではCOMポートの1つ、Linuxでは任意の数のデバイスファイルとして表現されます。ただし、LinuxではデバイスがUSB接続されている場合、usbmon カーネルモジュールを使用してUSB経由でシリアルデータをキャプチャできます。

3.1. usbmon をロードする

ターミナルで実行 `sudo modprobe usbmon`

入力したパスワードは、`sudo` 入力時に画面に何も表示されません

? 制御ユーザーのパスワードを覚えていますか?

物事

✓ 何も知らせないのは良い知らせです!

コマンドやパスワードに関するエラーが表示されず、他の出力なしでコマンドプロンプトに戻れば、成功したことを意味します

確実にロードされたことを確認したい場合は、実行して `lsmod` リストに含まれているかどうかを確認できます。

3.2. PLCとの通信

フィールドデバイスは、多くの場合、異なるネットワーク媒体を介して通信を行います。通信といえばイーサネットを思い浮かべる人が多いですが、多くのデバイスは、シリアルポート、パラレルポート、USBポートなど、他の通信媒体をサポートするために追加の接続を備えています。これらのポートを介して交換される情報は通常、デバイスとマスターサーバーまたは管理ソフトウェアがサポートする特定の産業用プロトコルを使用します。このラボでは、Velocio PLCをControlThings VMIに接続し、いくつかの簡単な手法を用いて、USBポート経由で送信されるModbus RTUシリアルトラフィックを監視します。

PLCとインターフェースを取り、いくつかのコマンドを実行するスクリプトを作成しました。スクリプト名はです `velocio-lab`。このスクリプトをというLinuxユーティリティで実行します `watch`。この `watch` コマンドは2秒ごとにスクリプトを実行し `velocio-lab`、実行されるたびにターミナルの出力を更新します。これにより、コンピュータとPLC間の通信が維持され、両者間の通信をキャプチャできるようになります。

ターミナルで実行 `watch velocio-lab`

✓ 成功

```
Input: RunProcess = True
Input: EmergencyStop = False
Output: RunIntakePump1 = False
Output: RunIntakePump2 = False
Output: RunIntakePump3 = True
Output: RunMixer = True
Output: RunOuttakePump = False
Output: EmergencyFlush = False
Holding Register: State = 3
Holding Register: Timer = 4094
```

このプログラムは、USBチャンネルを介してPLCにModbus-RTU `velocio-lab`を送信するPythonスクリプトです。ターミナルでは、2秒ごとに出力が変化し、PLCプロセスの実行中にPLCのタグ値がリアルタイムで更新されます。これはテキストベースのHMIと考えてください。15秒間にPLCプロセスの5つの状態を進むにつれて、**状態タグとタイマータグの値が変化することを見ることができます**。タイマータグは秒数をミリ秒単位で表示するため、4094は4.094秒です `velocio-lab`

3.3. WiresharkでUSBをキャプチャする

usbmon がロードされ、velocio-lab スクリプトが PLC と通信するようになったので、Wireshark を使用してトラフィックをキャプチャして分析できます。

≡ アクティビティメニューからWiresharkを開きます

最も簡単な方法は、[アクティビティ]メニューをクリックすると表示される検索ボックスに「wireshark」と入力して検索することです。

Wiresharkが開くと、キャプチャするインターフェースを指定する画面が表示されます。ウィンドウ中央の「**キャプチャ**」セクションには、複数のデバイスが表示されます。このセクションのスクロールバーを使うと、複数のUSBMONデバイスが表示され、それ

それ末尾に異なる番号が付いているのが確認できます。

USBモニタデバイスについて

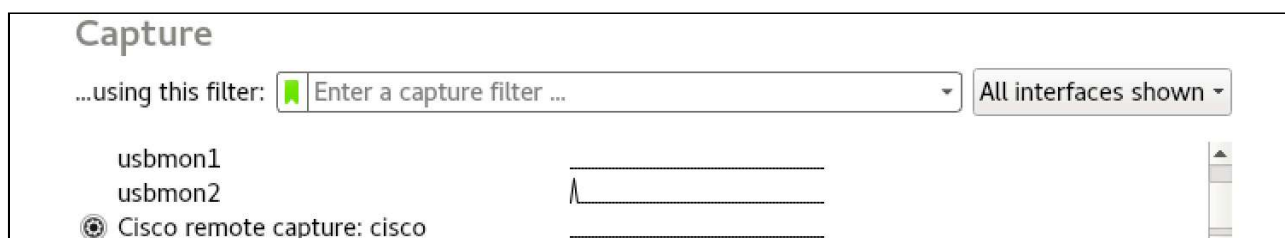
マザーボードに搭載されている内蔵USBハブごとに、USBモニタデバイスが1つあります。番号は1から始まり、そこから増加します。最近のコンピュータのほとんどは約4つの内蔵USBハブを備えているため、usbmon1、usbmon2、usbmon3、usbmon4という番号が表示されることが多いでしょう

すべてのシステムには、内部USBバスの集合を表す特別なusbmon0デバイスも存在します。このデバイスをキャプチャすることは推奨されません。必要なUSBトラフィックを見つけるのが難しくなる場合が多く、Wiresharkでは強制プロトコルデコードに関して制限があるためです。

usbmonからキャプチャを開始#

前の手順でメモしたUSBバス番号を覚えていますか？ここで必要になります。メモしたUSBバス番号を表す**usbmon#**をダブルクリックしてください。

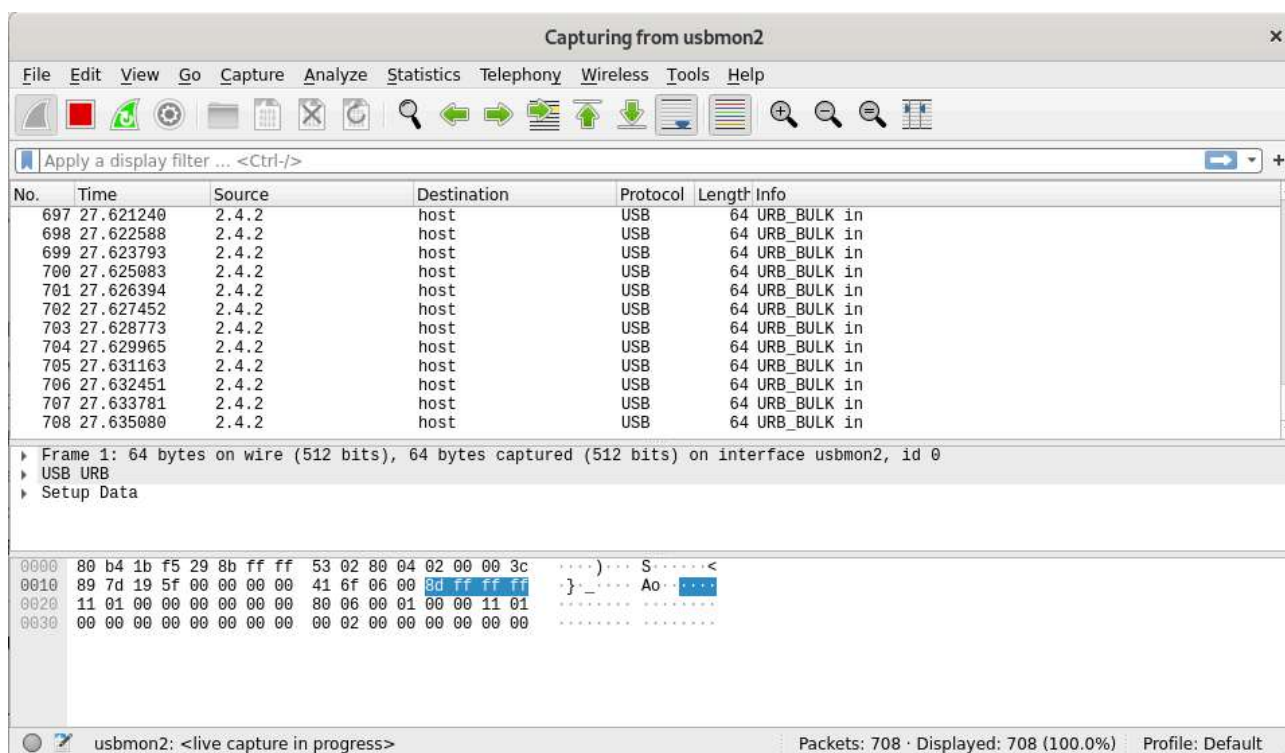
ヒント



このスクリーンショットでわかるように、Wiresharkは現在トラフィックのあるキャプチャデバイスを示すインジケータも表示します

成功

画面に2秒ごとにパケットのバーストが表示されるはずです。これは、velocio-labスクリプトの間隔と一致しているはずです



4〜5秒後にキャプチャを停止します

Wireshark ツールバーの赤い四角形のアイコンをクリックすると、キャプチャを停止できます。

velocio-lab これで、PLC と通信するスクリプト の完全なパケット キャプチャが少なくとも 2 つ取得されるはずです。

4. USBシリアルトラフィックの解析

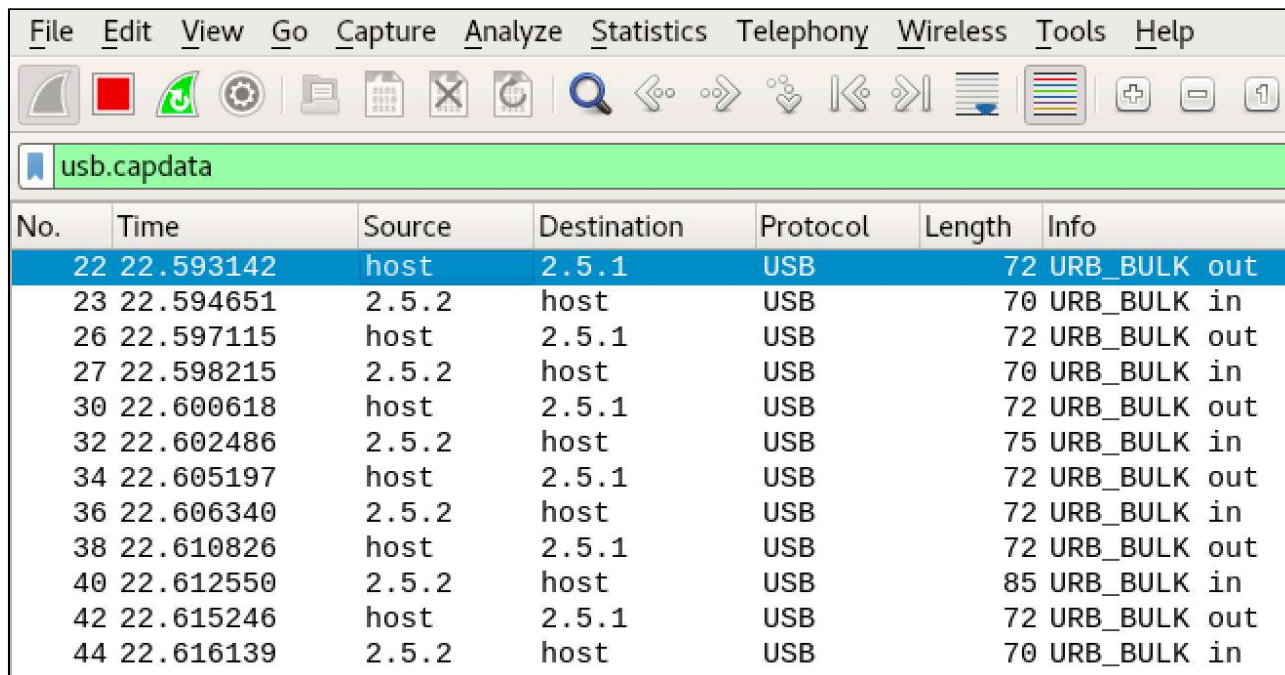
ネットワークパケットのリストが表示されます。Wiresharkを初めて使用する場合は、ウィンドウ上部にメニューとツールアイコンが表示されます。これらのアイコンの下にはフィルターバーがあります。Wiresharkのフィルター言語に慣れると、独自のフィルターを簡単に入力して、Wiresharkでパケットキャプチャのサブセットを表示できるようになります。フィルターバーの下には、Wiresharkの最も重要な部分である、キャプチャしたすべてのネットワークパケットの詳細を表示するテーブルがあります。これらのいずれかをクリックしてパケットを選択すると、ウィンドウの下部にある2つの部分が変わり、そのパケットの詳細が表示されます。最初の詳細ウィンドウには、各プロトコルヘッダーとそのコンポーネントの内訳が人間が読める形式で表示されます。その下には、16進数とASCIIの両方で表示された生のパケットがあります。

スクリプトを実行するたびに大量のパケットが生成されていることに気づくでしょう **velocio-lab**。これらのパケットのほとんどは、USB制御トラフィック、シリアルチャネルの確立、送信データの確認応答、そしてシリアルチャネルの切断です。これらのUSB制御トラフィックをすべてフィルタリングし、シリアルチャネル経由で送信されるModbus-RTUトラフィックに集中できるようにします。

usb.capdata Wiresharkフィルターを入力して Enter

これにより、シリアルチャネルデータ以外のすべてがフィルタリングされます。

✓ 成功



The image shows the Wireshark network protocol analyzer interface. The filter bar at the top contains the text 'usb.capdata'. Below the filter bar, a list of captured packets is displayed. The list is filtered to show only USB-related traffic. The columns shown are No., Time, Source, Destination, Protocol, Length, and Info. The packets are numbered 22 through 44. The 'Info' column shows details like '72 URB_BULK out' or '70 URB_BULK in'.

No.	Time	Source	Destination	Protocol	Length	Info
22	22.593142	host	2.5.1	USB	72	URB_BULK out
23	22.594651	2.5.2	host	USB	70	URB_BULK in
26	22.597115	host	2.5.1	USB	72	URB_BULK out
27	22.598215	2.5.2	host	USB	70	URB_BULK in
30	22.600618	host	2.5.1	USB	72	URB_BULK out
32	22.602486	2.5.2	host	USB	75	URB_BULK in
34	22.605197	host	2.5.1	USB	72	URB_BULK out
36	22.606340	2.5.2	host	USB	72	URB_BULK in
38	22.610826	host	2.5.1	USB	72	URB_BULK out
40	22.612550	2.5.2	host	USB	85	URB_BULK in
42	22.615246	host	2.5.1	USB	72	URB_BULK out
44	22.616139	2.5.2	host	USB	70	URB_BULK in

パケットリストの左側の列を見てください。ほとんどのパケット番号がフィルタリングされていることがわかります

フィルタリングされたビューの最初のキャプチャをよく見てみましょう。パケットリストの下にあるパケットデコードツリービューでは、パケットの詳細な内訳を確認できます。これらのパケットのそれぞれに「」という行があることに注目してください **Leftover Capture Data**。これはUSB URBチャネルを介してプッシュされるシリアルトラフィックです。Wiresharkでは、このデータを列として追加することで、メインのパケットビューに各パケットのデータを表示できます。

列として適用

Leftover Capture Data パケットデコードツリービューの行を右クリックします

選択 Apply as Column

✓ 成功

Leftover Capture Data	Info
010200000006f808	URB_BULK out
010201016048	URB_BULK in
0101000a00069c0a	URB_BULK out
01010108504e	URB_BULK in
01030000000305cb	URB_BULK out
010306000426a600003bde	URB_BULK in

これで、メインパケットリストで各USBパケットの生のシリアルデータを確認できるようになります

ヒント

Wiresharkのこのトリックを知らなかった方は、ぜひ覚えておいてください。これは、Wiresharkで一連のパケット全体で何が起きているかを分析する最も簡単な方法であり、詳細セクションのほぼすべての行で実行できます

4.1. Modbus-RTU解析

Modbus-RTUトラフィックを詳しく見てみましょう。

ヒント

Info 右側の列を使用すると、**out** velocio-labスクリプトからのリクエストパケットと、**in** PLCからのレスポンスパケットを確認できます

Leftover Capture Data	Info
010200000006f808	URB_BULK out
010201016048	URB_BULK in
0101000a00069c0a	URB_BULK out
01010108504e	URB_BULK in
01030000000305cb	URB_BULK out
010306000426a600003bde	URB_BULK in

この Modbus-RTU REQUEST をデコードしやすくするために、Modbus RTU パケットを構成するさまざまなフィールドを以下に示します。

ユニットID	読み取り機能	開始アドレス	読み取り番号	CRC
1バイト	1バイト	2バイト	2バイト	2バイト
(16進数2文字)	(16進数2文字)	(16進数4文字)	(16進数4文字)	(16進数4文字)

2番目のフィールド「Read Function」では、異なるModbus機能を表すために異なる番号が使用されます。Modbusでよく見られる4つの基本的な読み取り機能は次のとおりです。

- 01 – コイル（デジタル出力と内部ブールタグ）の読み取り
- 02 – 離散入力（デジタル入力とも呼ばれる）を読み取る
- 03 – 保持レジスタ（アナログ出力と内部の非ブールタグ）の読み取り
- 04 – 入力レジスタ（アナログ入力とも言う）の読み取り

これらの関数定義を使用して、上記のスクリーンショットに示されている3つのModbus RTUリクエスト（1行目、3行目、5行目）を手動でデコードできますか？これらは、VMのWiresharkに表示される内容と一致するはずです。

以下に生のリクエストをコピーし、各プロトコルフィールドの間にスペースを追加してあります。偶数行はレスポンスであり、別の方法でデコードする必要があるため、ここでは省略します。

最初のリクエストは何ですか？（1行目）

01 02 0000 0006 f808

成功

ユニット1から、ディスクリート入力を読み取ります（機能2）。アドレス0から開始し、6つのアドレス（すべてのPLC物理入力）を読み取ります。
これにより、物理入力1〜6に対応するアドレス0〜5が読み取られる。

2番目のリクエストは何ですか？（3行目）

```
01 01 000a 0006 9c0a
```

✓ 成功

ユニット1から、コイル（機能1）を読み取ります。アドレス10（16進数で000a）から開始し、6つのアドレス（すべてのPLC物理出力）を読み取ります

これにより、物理出力1～6に対応するアドレス10～15が読み取られる。

3番目のリクエストは何ですか？（5行目）

```
01 03 0000 0003 05cb
```

✓ 成功

ユニット1から、保持レジスタ（機能3）を読み取り、アドレス0から3つのアドレスを読み取る

これは、状態タグ（アドレス0）とタイマータグ（アドレス1～2）になります。

Modbusアドレスがどのように割り当てられているかを確認したい場合は、Windows 10 VMに戻り、PLC用のvBuilderロジックを開いてください。プログラムを開くと、上部にバスのようなボタンがあり、「MOD」の文字が書かれています。分かりますか？Mod-Busですね... :) これをクリックすると新しいウィンドウが開き、任意のModbusアドレスにタグを割り当てることができます。

Modbus-RTU 応答を手動でデコードすることに興味がありますか？以下のチャレンジでヒントをいくつかご紹介します。

ちなみに、「残りのキャプチャデータ」列は、右クリックして「この列を削除」を選択することでいつでも削除できます。

また、ターミナルウィンドウを閉じて、velocio-labスクリプトがPLCとの通信を停止するようにする必要があります。

5. ボーナスチャレンジ

他の参加者のラボの完了を待っている場合、またはこのラボで紹介された概念をさらに深く探求したい場合は、以下のアイデアを試してみてください。これらのチャレンジにはチュートリアルは用意されていませんが、行き詰まった場合は、コース作成者のジャスティンに取り組みの過程を見せていただければ、喜んでヒントをくれるでしょう。justin_@controlthings.ioまでメールでお問い合わせください

5.1. デコード形式

デコード済みツリービューの「残りのキャプチャデータ」フィールドの1つを右クリックし、「デコード形式」を選択してください。このフィールドでModbusデコードを強制する方法がわかりますか？警告：この変更を行った後は、Wiresharkフィルターを変更することを忘れないでください。「残りのキャプチャデータ」はなくなります

5.1. Python で遊ぶ

非常に短くて分かりやすいスクリプトをご覧ください。スクリプトの場所は以下を実行することで確認できます。 `which`
`velocio-lab`

5.2. Modbus-RTU 応答のデコード

フォルダ内を見ると `Samples/Protocols/Modbus`、プロトコル規格が見つかります。それを使って、キャプチャファイル内の Modbus-RTU 応答パケットをデコードしてみてください。